

PART 01

1. Why is it recommended to explicitly assign values to enum members in some cases?

Answer: Explicitly assigning enum values makes their numeric values predictable and stable. This is important when enum values must match database values, external systems, APIs, or predefined business rules. It also prevents unexpected changes if new members are added later.

2. What happens if you assign a value to an enum member that exceeds the underlying type's range?

Answer: The enum member cannot be represented by the selected underlying type. As a result, the value assignment causes a compile-time error because it is outside the valid range of that type.

3. What is the purpose of the virtual keyword when used with properties?

Answer: The virtual keyword allows a property to be overridden in a derived class. This supports polymorphism, because a derived class can provide its own implementation of the property while the property is accessed through a base-class reference.

4. Why can't you override a sealed property or method?

Answer: The sealed keyword prevents a member from being overridden again in derived classes. It is used when the current implementation should remain fixed and inheritance should not change that behavior.

5. What is the key difference between static and object members?

Answer: Static members belong to the class itself and are shared by the type, so they can be accessed without creating an object. Object or instance members belong to a specific object, and each object can have its own instance data.

6. Can you overload all operators in C#? Explain why or why not.

Answer: No. C# allows overloading only for operators that the language defines as overloadable. Some operators cannot be overloaded because their behavior is controlled directly by the language and its syntax.

7. When should you consider changing the underlying type of an enum?

Answer: You may change the underlying type when the enum values fit in a smaller numeric type and reducing memory usage is useful. It can also be necessary when the enum must match a specific numeric representation required by another system.

8. Why can't a static class have instance constructors?

Answer: A static class cannot be instantiated, so there is no object that needs to be initialized by an instance constructor. Therefore, normal instance constructors are not allowed in a static class.

9. What are the advantages of using Enum.TryParse over direct parsing with int.Parse?

Answer: Enum.TryParse safely attempts to convert a string into an enum value and indicates success or failure without throwing an exception for invalid input. This makes input validation easier and safer.

10. What is the difference between overriding Equals and == for object comparison in C# struct and class?

Answer: Equals is a method used to define logical equality and can be overridden to compare meaningful data. The == operator follows its defined operator behavior; for classes it normally compares references unless overloaded, while structs do not automatically have a field-based == operator.

11. Why is overriding ToString beneficial when working with custom classes?

Answer: The default ToString usually returns the type name. Overriding it allows the class to return a useful representation of its data, which makes debugging, logging, and displaying objects much easier.

12. Can generics be constrained to specific types in C#? Provide an example.

Answer: Yes. Generic type parameters can be restricted using constraints. For example, 'where T : class' requires T to be a reference type, while 'where T : BaseClass' requires T to inherit from BaseClass.

13. What are the key differences between generic methods and generic classes?

Answer: A generic method defines type parameters for a specific method, while a generic class defines type parameters for the whole class. The class type parameter can therefore be used by its fields, properties, and multiple methods.

14. Why might using a generic swap method be preferable to implementing custom methods for each type?

Answer: A generic swap method can work with many different data types using one implementation. This reduces duplicated code, improves reusability, and keeps the code type-safe.

15. How can overriding Equals for the Department class improve the accuracy of searches?

Answer: It allows Department objects to be compared according to meaningful business data, such as an ID or name, instead of only checking whether they are the same object reference. This makes searches more logically accurate.

16. Why is == not implemented by default for structs?

Answer: C# does not automatically define a == operator that compares all fields of a struct. Therefore, if direct operator-based equality is required for a custom struct, the == operator must be explicitly overloaded.

PART 02

1. What do we mean by Generalization concept using Generics?

Answer: Generalization using generics means creating reusable code that can work with different data types through type parameters. Instead of writing separate implementations for each type, one generic implementation can be used safely with many types.

2. What do we mean by hierarchy design in real business?

Answer: Hierarchy design means organizing business entities into levels based on shared characteristics and relationships. A general entity can contain common features, while more specific entities can extend it with specialized behavior.

PART 03 BONUS

1-SELF-STUDY

1. What is the purpose of a partial class in C#?

Answer: A partial class allows the definition of one class to be divided into multiple parts, usually across different files. All parts are combined by the compiler into one class. This is useful for organizing code and separating different parts of a large class.

2. What is the readonly keyword in C#?

Answer: The readonly keyword is used with fields whose value should not be reassigned after initialization. A readonly field can be assigned when it is declared or inside a constructor, but it cannot normally be assigned again after that.

3. What are compound operators in C#?

Answer: Compound assignment operators combine an arithmetic or other operation with assignment. Examples include +=, -=, *=, and /=. For example, x += 5 means that 5 is added to the current value of x and the result is assigned back to x.

4. What is the difference between const and readonly?

Answer: const represents a compile-time constant and must be assigned when it is declared. readonly is determined at runtime and can be assigned in the declaration or in a constructor. A readonly field can therefore have different values in different objects.

5. What is a private constructor?

Answer: A private constructor is a constructor that can only be accessed from inside its own class. It prevents other classes from creating instances directly and allows the class to control object creation.

6. How is a private constructor used in the Singleton design pattern?

Answer: In the Singleton pattern, the private constructor prevents external code from creating multiple objects directly. The class controls access to its single shared instance, usually through a static member.

2- What is event-driven programming?

Answer: Event-driven programming is a programming style in which the program responds to events such as mouse clicks, keyboard input, messages, or other notifications. When an event occurs, the related handler executes the required logic.